API SECURITY RISK ANALYSIS REPORT

Target: DummyJSON public demo API (https://dummyjson.com)
Prepared by: Edwin Baiju
Cyber Security Internship, Future Interns (Task 3)
Date: October 2026

1. EXECUTIVE SUMMARY

APIs are the connections that let apps and websites exchange data. If an API is weak, attackers can read private data, take over accounts or overload a service. This report reviews a public demo API (DummyJSON, built for testing and filled with fake data) for common API security problems.

The review found 4 issues: 1 High, 1 Medium and 2 Low, along with several good practices. The most serious issue is that user records, including names, emails, phone numbers and addresses, can be read without logging in. Rate limiting was also not visible. These issues are treated as they would be in a real business system.

Overall risk rating: HIGH (in a real system)

2. SCOPE AND METHOD

Target: DummyJSON demo API, a public API made for practice, with fake data.
Tools: Postman (no account), browser.
Approach:
1. Sent normal requests to public endpoints and reviewed the data returned
2. Checked response headers for security settings
3. Logged in with DummyJSON's published demo account and tested protected endpoints with and without a token
4. Sent about 20 requests to one endpoint to check for rate limiting
5. Sent empty and unusual login data to check input validation
Limits: Only light, normal requests were used. No attacks, floods or stress tests were run, and only fake demo data was involved.

3. RISK RATING GUIDE

High: could lead to stolen data or account takeover; fix immediately.
Medium: makes attacks easier; fix soon.
Low: minor weakness; fix when possible.

4. SUMMARY OF FINDINGS

High: 1
Medium: 1
Low: 2

5. DETAILED FINDINGS

FINDING 1: Excessive Data Exposure and Weak Access Control on User Records
Risk: HIGH (in a real system)

What it is: User records can be read by anyone, with or without a token, by using the user list or changing the ID in the address (for example /users/2). Each record includes name, email, phone, address and birth date.
Business impact: Attackers could collect customer personal data in bulk, leading to privacy breaches, phishing and legal trouble.
Fix: Let users read only their own record, check ownership on the server for every request, and return only the fields that are needed.
Evidence: Figure 1
Note: The data here is fake. The rating shows how this would look in a real system.

FINDING 2: No Visible Rate Limiting
Risk: MEDIUM
What it is: 20 rapid requests to the same endpoint all returned 200 OK, with no 429 (Too Many Requests) response.
Business impact: Without limits, attackers can try thousands of passwords, copy all data or overload the service.
Fix: Add rate limits per user and per IP address, return 429 when exceeded, and use stricter limits on login.
Evidence: Figure 2
Note: Only 20 requests were sent, so a higher limit may exist.

FINDING 3: Content-Security-Policy Header Not Set
Risk: LOW
What it is: API responses do not include a Content-Security-Policy header.
Business impact: CSP helps browsers block injected scripts. It matters most for web pages, so for a JSON API this is a minor hardening gap.
Fix: Add a restrictive header such as default-src 'none' to API responses.

FINDING 4: Verbose, Detailed Responses
Risk: LOW
What it is: Responses return many fields per record, including details that a client likely does not need.
Business impact: Extra data increases what is exposed if any other weakness is found.
Fix: Return only the fields each client needs.

GOOD PRACTICES OBSERVED
- Protected endpoint (/auth/me) rejected requests with no token (401 Unauthorized).
- No open CORS policy: the API does not send Access-Control-Allow-Origin: *, so other websites cannot read its responses from a browser.
- Input validation: an empty login returned 400 "Username and password required".
- Special characters (emilys'--) in a login returned 400 "Invalid credentials", with no server error or technical details (Figure 3).

6. RECOMMENDED ACTION PLAN
Fix first: restrict access to user records and return only needed fields.
Fix next: add rate limiting, with stricter limits on login.
Ongoing: add security headers, review API responses regularly, and repeat the assessment after fixes.

7. CONCLUSION

The API handles authentication and input checking well, but exposes too much user data and shows no rate limiting. These are common API weaknesses and can be fixed with access checks and simple server settings. After the fixes, the API should be tested again.

Note: The target is a public demo API with fake data. Only light, normal requests were used.

EVIDENCE

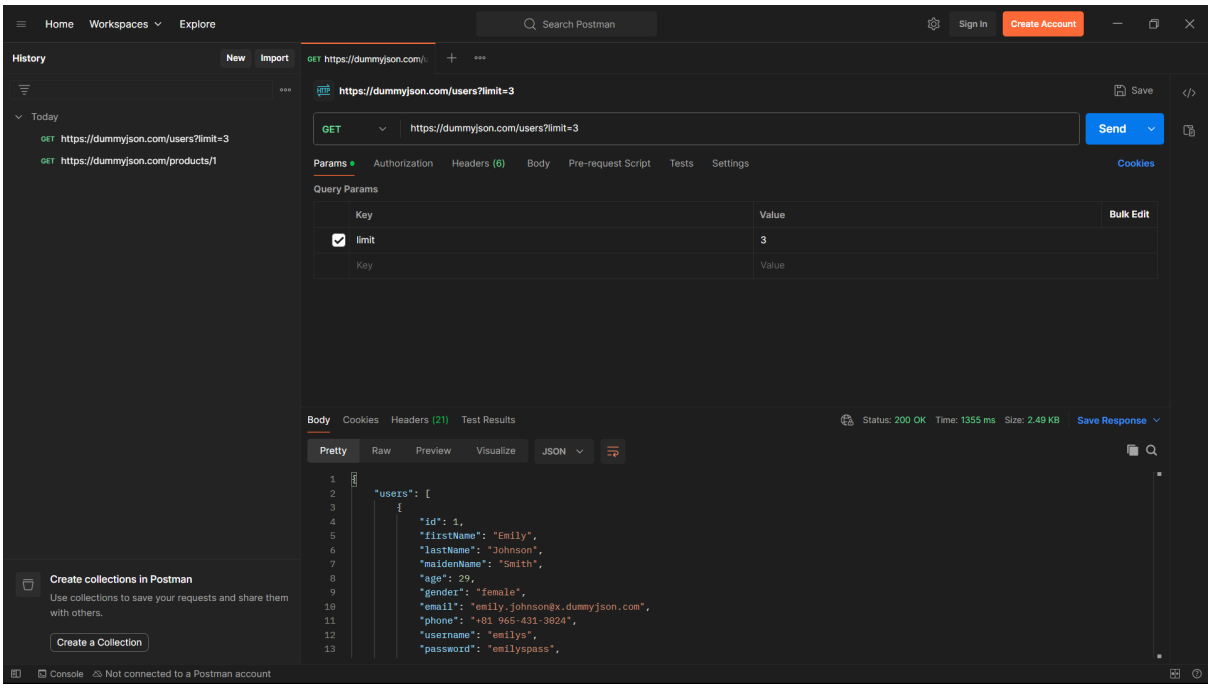


Figure 1: User records returned without logging in

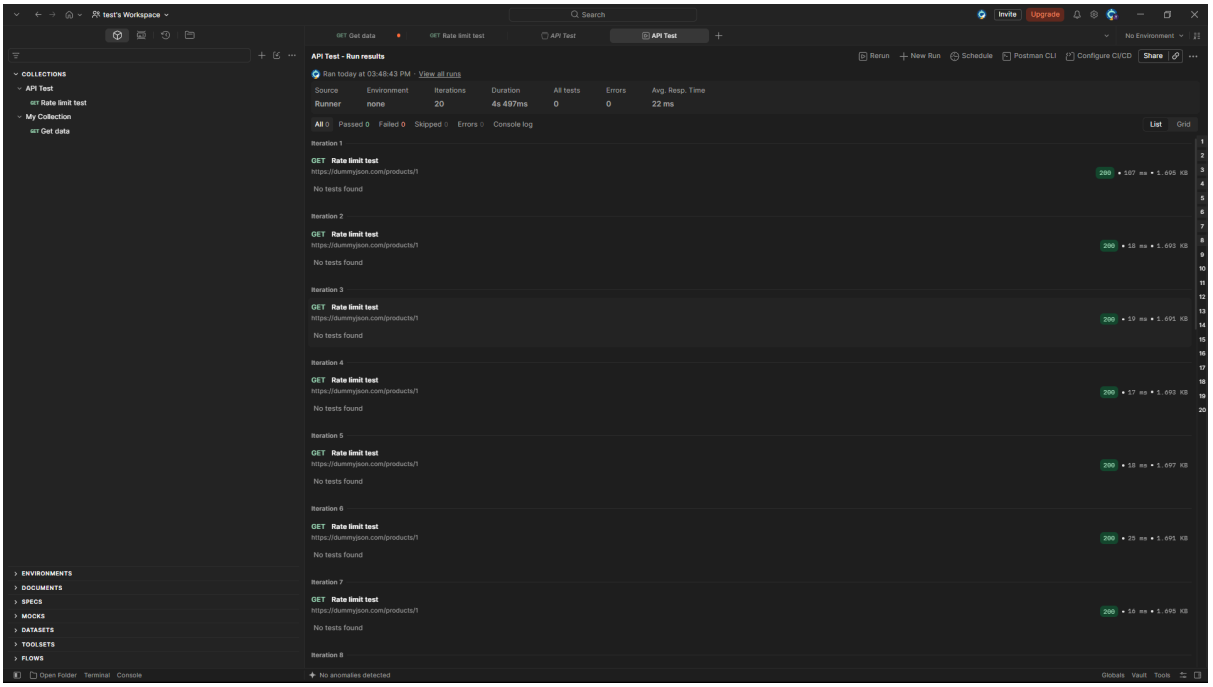


Figure 2: 20 requests all returned 200 OK

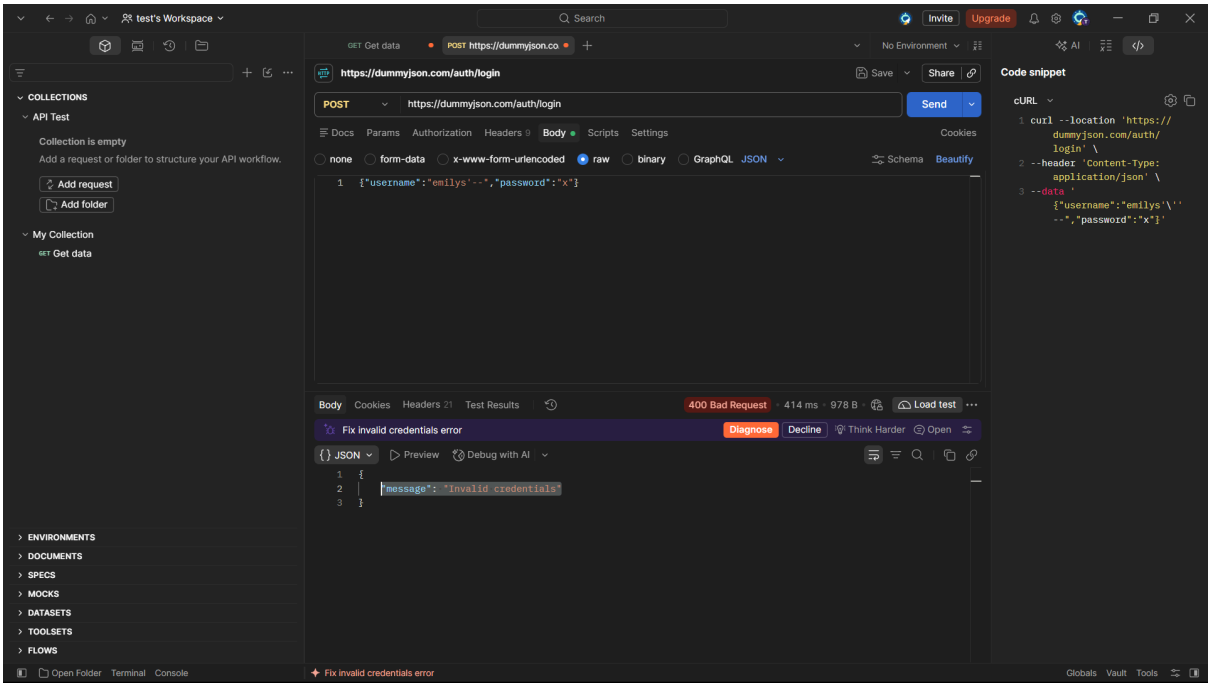


Figure 3: Login with unusual input returned 400 Invalid credentials